

Humans versus AI: A Multi-Dimensional Characterization of Code Quality and Security

TODO Author 

TODO Affiliation, Country

Abstract

AI coding assistants now reach over 84% of developers, yet the software engineering community still lacks a unified empirical picture of how AI-generated code compares to human-written code across quality and security dimensions. Existing studies fragment this picture by using different datasets and toolchains, making it hard to distinguish patterns that hold generally from those that are artifacts of a particular setup. To address this, we analyse 109,993 source files from the CodeMirage benchmark, covering 10 programming languages and 10 production-level LLMs alongside 10,000 human-authored files, examining stylistic violations, structural complexity metrics, security vulnerability profiles, and software maintainability indicators, with all comparisons grounded in nonparametric statistical tests and effect sizes. The results reveal that AI-generated code carries $4.6\times$ higher violation densities overall, though the language's type discipline strongly moderates this gap: statically typed languages narrow it to $1.32\times$ while dynamically typed languages widen it to $2.75\times$. Structurally, AI-generated code pairs higher cyclomatic complexity with lower class coupling and reduced lexical diversity, a profile that suggests conventional complexity-quality assumptions do not transfer to AI-generated code the way they do to human-written software. Security vulnerability distributions follow distinct patterns across origin, and when comparing open-weight and closed-weight LLMs on the same benchmark, measurable differences emerge in both quality and security profiles, indicating that model weight accessibility is a meaningful axis of variation. Taken together, these findings offer an empirical baseline that can inform how generation models are evaluated, how developers are trained to supervise AI output across different language contexts, and how detection and attribution tools are built on reproducible evidence.

2012 ACM Subject Classification Software and its engineering $\rightarrow$ Software defect analysis; Software and its engineering $\rightarrow$ Software testing and debugging

Keywords and phrases TODO: comma-separated keywords

Digital Object Identifier 10.4230/LIPICs.TODO.2025.TODO

Funding *TODO Author*: TODO funding

Acknowledgements TODO: Acknowledgments text. Include funding sources, data providers, or reviewers as appropriate.

1 Introduction

AI-assisted programming tools have become a standard part of modern software development. Systems such as GitHub Copilot, Devin, and Cursor now automate tasks ranging from code completion to pull request creation, operating with minimal human oversight. The scale of this adoption is substantial: the AIDev dataset documents over 980,000 AI-generated contributions spanning more than 116,000 repositories [?]. We draw on this dataset to examine how AI authorship affects code quality across a broad range of open-source projects.

While automation accelerates development, it also introduces quality concerns that differ in character from those typically associated with human-authored code. Existing research shows that AI-generated code exhibits higher error rates than human-authored code [?, ?, ?], yet several practical questions remain unanswered: (1) which violation categories and severity levels predominate in AI-generated code, (2) how structural complexity metrics relate to



© TODO Author Names;
licensed under Creative Commons License CC-BY 4.0
TODO Conference Name (TODO 2025).

Editors: TODO; Article No. TODO; pp. TODO:1–TODO:14



Leibniz International Proceedings in Informatics
Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl Publishing, Germany

defect density and whether these relationships differ between AI and human-authored code, and (3) which programming languages face the largest quality gaps and whether static type systems reduce AI-associated risks.

We address these questions through a large-scale empirical investigation of 19,539 files drawn from the AIDev dataset, applying three complementary analyses: language-specific linters to assess functional correctness, structural complexity measurements to assess architectural quality, and security scanners to characterize CWE patterns. All analyses are stratified by programming language, enabling direct comparison of AI-generated and human-authored code across multiple quality dimensions. The main contributions of this work are as follows:

- 1. Large-scale empirical assessment of AI-generated versus human-authored code quality.** We analyze 19,539 files across six programming languages using a multi-dimensional framework covering functional, structural, and security quality dimensions, enabling systematic characterization of quality differences across language families and authorship types.
- 2. Evidence that type safety reduces AI-associated quality gaps.** Statically-typed languages (Java, C#, TypeScript) exhibit significantly narrower violation density gaps between AI-generated and human-authored code (1.32x to 1.66x) compared to the 2.34x gap observed in dynamically-typed languages. Chi-square analysis confirms this pattern ($p < 0.001$), suggesting that compiler-enforced type checking intercepts a class of errors that otherwise propagate in dynamic languages.
- 3. Identification of divergent structural complexity patterns in AI-generated code.** We provide empirical evidence that AI-generated code exhibits an ‘inverted’ quality profile characterized by high cyclomatic complexity alongside low structural coupling, a pattern statistically distinct from human-authored code ($p < 0.05$). This finding indicates that the structural composition of AI-generated code does not conform to assumptions underlying conventional quality metrics.

2 Related Work

2.1 TODO: Subsection Title

TODO: Synthesize prior work thematically. Compare findings across studies and identify unresolved gaps. Separate pre-AI era findings from AI era findings where relevant.

2.2 TODO: Subsection Title

TODO: Position the current work relative to the closest prior studies. Highlight what this paper addresses that prior work does not.

3 Experiment Design

We characterize defect patterns across three analytical dimensions: functional correctness, structural complexity, and security vulnerabilities.

3.0.1 Notes

A statistically significant regression slope ($p < 0.05$) is interpreted as evidence of a trend. In cases where the data may violate linearity or normality assumptions, we apply the nonparametric Mann-Kendall trend test [?], which is robust to non-normal distributions and

outliers. This combined methodology ensures the reliability and validity of our assessment of issue prevalence trends across heterogeneous language and authorship combinations.

3.1 RQ1: Functional Correctness and Defect Prevalence

RQ1: *Does AI-generated code exhibit different patterns of linter violations and defect types compared to human-written code across various programming languages?*

Using the linter toolchain described in §4.2, we compute violation rates per file and test statistical significance using chi-square tests (χ^2 , $\alpha = 0.05$) with Cohen’s h effect sizes. We break down violations by severity level and identify top violation messages to characterize predominant issue types. Finally, we stratify by language type, comparing statically typed languages (C, C++, C#, Java, Go) against dynamically typed languages (Python, JavaScript, PHP, Ruby) to test whether static type systems mitigate defect risks in AI-generated code. HTML is analyzed separately as a markup language and excluded from this typed-language comparison.

3.2 RQ2: Structural Complexity and Code Composition

RQ2: *Do structural characteristics such as complexity, coupling, and lexical volume differ statistically between AI-generated and human-written code?*

Traditional software quality models assume structural complexity metrics predict defect density. We test whether this assumption generalizes to AI-generated code using the 27 structural metrics described in §4.2. We compute Pearson correlations between each metric and violation counts separately for AI and human samples, then apply Fisher’s z-transformation to test whether correlation strengths differ significantly between authorship types ($\alpha = 0.05$, Bonferroni-corrected). This analysis tests whether AI code breaks traditional complexity-quality relationships through unexpected patterns such as high control-flow complexity paired with low architectural coupling.

3.3 RQ3: Security Vulnerability Patterns

RQ3: *What security vulnerability patterns are most prevalent in AI-generated code, and how do they vary by language?*

Security vulnerabilities pose direct exploitation risks distinct from general correctness issues. Using the Semgrep analysis protocol described in §4.2, we extract CWE identifiers and severity levels (info, warning, error, critical) for each finding. We compute vulnerability density separately for AI and human code, then apply Mann-Whitney U tests and Cliff’s delta to test whether differences are significant ($\alpha = 0.05$, Bonferroni-corrected). We specifically examine: (1) overall vulnerability density differences between authorship types, (2) which CWE categories dominate AI-generated code, and (3) whether statically typed languages exhibit lower vulnerability rates than dynamically typed languages.

4 Methodology

4.1 Dataset and Data Processing

We use the CodeMirage benchmark [?], a publicly available multilingual dataset hosted on Hugging Face that provides paired human-written and AI-generated source code across ten programming languages: C, C++, C#, Go, HTML, Java, JavaScript, PHP, Python, and Ruby. CodeMirage comprises 10,000 human-written files and approximately 99,993

126 AI-generated counterparts produced by ten production-level LLMs spanning six providers.
 127 Table 1 summarizes the language-level distribution.

128 The human-written code was drawn from the CodeParrot GitHub-Code-Clean dataset [?],
 129 a curated subset of publicly available GitHub repositories sanitized prior to May 2022, before
 130 the widespread deployment of code-generating LLMs and autonomous coding agents. This
 131 temporal bound ensures that the human baseline is free from AI contamination. For each
 132 language, 1,000 files were sampled with additional length-based filtering to preserve diversity
 133 while bounding file size to a tractable scale for static analysis.

134 The AI-generated counterparts were produced using a two-stage text-to-code strategy.
 135 Each human-written file was first summarized by an LLM, capturing its purpose, functionality,
 136 logic structure, and key identifiers. A separate LLM was then prompted to generate a
 137 functionally equivalent implementation from the summary alone, without access to the
 138 original source. This approach prevents direct recitation while preserving functional intent.
 139 A rule-based inspector enforced that each generated file remained within acceptable bounds
 140 of the original’s line count and character length, and required a BLEU score below 0.5 to
 141 prevent near-verbatim copying. Samples failing repeated generation attempts were discarded.

142 The ten LLMs contributing AI-generated code are Claude-3.5-Haiku, GPT-4o-mini,
 143 GPT-o3-mini, Gemini-2.0-Flash, Gemini-2.0-Flash-Thinking, Gemini-2.0-Pro, DeepSeek-V3,
 144 DeepSeek-R1, Llama-3.3-70B, and Qwen-2.5-Coder-32B [?]. For our quality characterization,
 145 we aggregate all AI-generated files across the ten LLMs into a single AI-authored category,
 146 allowing us to characterize properties of AI-generated code broadly rather than attributing
 147 findings to any specific model.

148 We restrict our structural complexity analysis (RQ2) to the nine compiled or interpreted
 149 programming languages, excluding HTML. Metrics such as cyclomatic complexity and
 150 MaxInheritanceTree are defined for programming language constructs and are not meaningful
 151 for HTML markup. All other analyses include HTML where applicable tooling is available.

■ **Table 1** Dataset Distribution by Language

Language	AI Files	Human Files	Total
Python	10,000	1,000	11,000
Java	10,000	1,000	11,000
JavaScript	10,000	1,000	11,000
C++	9,999	1,000	10,999
C	10,000	1,000	11,000
C#	9,997	1,000	10,997
Go	10,000	1,000	11,000
PHP	9,998	1,000	10,998
Ruby	10,000	1,000	11,000
HTML	9,999	1,000	10,999
Total	99,993	10,000	109,993

152 **4.2 Static Analysis Toolchain**

153 To characterize qualitative differences between AI-generated and human-written code across
 154 multiple dimensions, we employ established static analysis tools to measure functional
 155 correctness, structural complexity, and security vulnerabilities.

4.2.1 Functional Defect Detection

For Python and Java, we follow the approach in Liu et al. [?], using Pylint [?] for Python and PMD [?] for Java. For JavaScript, we employ ESLint [?], a widely adopted linter for detecting syntax errors and style violations. For C and C++, we use Cppcheck [?], which captures a broad range of issues including undefined behavior, resource leaks, and portability concerns across both languages. For C#, we use Roslyn [?], integrated into the .NET SDK for syntax, semantic, and maintainability analysis. For Go, we apply staticcheck [?], a comprehensive static analysis tool that identifies correctness bugs, deprecated API usage, and performance issues specific to the Go language. For PHP, we use PHPStan [?], which detects type errors, undefined variables, and logic mistakes without code execution. For Ruby, we apply RuboCop [?], which checks for style violations, potential bugs, and complexity issues. For HTML, we use HTMLHint [?] to detect syntax errors and structural issues in markup files. Table 2 summarizes the linter coverage and violation counts across our dataset.

Table 2 Linter coverage and violation counts per language.

Language	Linter	# Files	# Violations
Python	Pylint	11,000	—
Java	PMD	11,000	—
JavaScript	ESLint	11,000	—
C	Cppcheck	11,000	—
C++	Cppcheck	10,999	—
C#	Roslyn	10,997	—
Go	staticcheck	11,000	—
PHP	PHPStan	10,998	—
Ruby	RuboCop	11,000	—
HTML	HTMLHint	10,999	—

4.2.2 Structural and Complexity Profiling

Existing works on program comprehension reveal that software metrics can be a valuable source of information for understanding the properties of a piece of software [?, ?, ?]. We use Understand by SciTools [?] to extract software metrics from the nine compiled and interpreted programming languages in our dataset, excluding HTML as noted in §4.1. Understand is an industry-standard tool for software analytics with support for all target languages. We extract 27 metrics per file capturing three key dimensions of code structure.

First, *Volume and Complexity* metrics including CountLineCode, CountStmt, and Cyclomatic Complexity quantify verbosity and control-flow density. Second, *Lexical Entropy* metrics use Halstead measures (Halstead Effort and Halstead Volume) to assess vocabulary richness and algorithmic complexity. Finally, *Architectural Depth* metrics such as CountClassCoupled and MaxInheritanceTree capture object-oriented coupling and inheritance patterns, applied to object-oriented languages where these constructs are defined (C++, C#, Java, PHP, Python, Ruby). These metrics enable analysis of whether traditional complexity-quality relationships established for human-written code generalize to AI-generated code.

We extract metrics at the file level to ensure consistent comparison across all languages and project structures. Metrics requiring cross-file dependencies or class hierarchy context are excluded, as they do not exhibit meaningful variation in file-level analysis. All metric extraction configurations and complete results are available in our replication package [?].

188 5 Results

189 5.1 RQ1: Defect Patterns and Categories

190 We compare linter-detected violation patterns in AI-generated and human-authored code
 191 across six programming languages. AI-generated code exhibits significantly higher violation
 192 rates overall, with the magnitude of the gap depending strongly on language typing discipline
 193 and the specific violation category examined.

194 5.1.0.1 Overall violation density.

195 A two-sample Mann–Whitney U test on per-file violation counts confirms that AI-generated
 196 files contain significantly more violations than human-authored files ($U = \dots, p < 0.001$).
 197 Cliff’s $\delta \approx 0.58$ indicates a medium-to-large effect, meaning that in more than half of all
 198 pairwise comparisons a randomly selected AI-generated file accumulates more violations than
 199 a randomly selected human-authored file. This gap is not driven by a few extreme outlier
 200 files; the distributional shift is consistent across the full sample.

201 5.1.0.2 Severity profile.

202 Higher violation counts would be less concerning if the excess violations were merely stylistic.
 203 To test this, we categorized all violations into three levels (Error, Warning, Info) and
 204 compared severity distributions with a chi-square test of independence. The two groups differ
 205 significantly ($\chi^2 = 24.73, df = 2, p < 0.001$, Cramér’s $V = 0.18$). AI-generated code carries
 206 a substantially higher proportion of error-level violations (32.5%) than human-authored
 207 code (18.7%), while human code shows a larger share of warning-level issues (61.2% vs.
 208 48.3% for AI). Error-level violations are those that can halt execution or indicate broken
 209 module dependencies; the near-twofold gap in error prevalence indicates that the elevated
 210 defect density in AI code is not cosmetic noise but includes a material share of functionally
 211 disruptive issues.

212 5.1.0.3 Language-level breakdown.

213 Table 3 reports per-file violation rates for each language. The differences are far from uniform.
 214 Python shows the largest gap: AI-generated files average 26.68 violations per file compared
 215 to 9.69 for human-authored files, a 2.75-fold increase. Python’s dynamically typed and
 216 interpreted nature means that type mismatches, undefined names, and import failures that
 217 would be caught at compile time in other languages persist as linter warnings, leaving more
 218 room for generation-time errors. TypeScript, by contrast, shows only a 1.32-fold difference,
 219 consistent with the expectation that static type checking constrains the space of valid code a
 220 model can generate.

221 C# presents an exception: AI-generated C# files have *fewer* violations per file (1.71)
 222 than human-authored files (3.93), a ratio of 0.43. C#’s strict compilation semantics may
 223 force AI models toward syntactically valid patterns, and the human baseline drawn from
 224 active pull requests may include draft-quality files with higher stylistic variation. Java’s
 225 human baseline records zero violations because those files were sourced from successfully
 226 merged pull requests subject to automated quality gates; the non-zero AI violation rate (4.99
 227 violations per file) reflects the gap between curated human contributions and unconstrained
 228 model output rather than a property of Java in general.

Aggregating by language family, statically typed languages (C#, Java, TypeScript) show a mean 1.66-fold gap, versus 2.34-fold for dynamically typed languages (Python, JavaScript). Static type systems reduce but do not eliminate generation-time quality differences.

5.1.0.4 Dominant violation categories.

Table 4 shows the two most frequently observed violations per language for each group. Several cross-language patterns emerge. Naming convention violations are the single most prevalent issue in AI-generated Python code (78.8% of files), reflecting systematic noncompliance with PEP8 identifier naming without project-specific configuration. Import errors appear in 56.5% of AI-generated Python files and 51.8% of human files, suggesting that import-related failures reflect dataset characteristics rather than an AI-specific failure mode. Unused variable warnings, however, occur in substantially more AI-generated JavaScript files (20.6%) than human files (8.3%), indicating that models generate variables that are declared but never consumed.

TypeScript shows a notable reversal: human-authored code has substantially higher parsing error rates (56.9% of files) than AI-generated code (14.2%). AI models, trained on large TypeScript corpora, produce syntactically more regular code, while human developers introduce stylistic diversity that linters flag more often. C# follows a similar pattern, with syntax errors more prevalent in human files (74.4%) than AI files (47.3%).

Java presents a stark contrast: AI-generated code produces violations at critical and medium severity levels, while the human baseline contains no violations at all. This reflects the quality selection effect of the merged-PR filter applied to the human sample rather than an inherent property of AI Java code.

Overall, RQ1 shows that AI-generated code accumulates more linter violations than human-authored code, with the widest gap in dynamically typed languages. The dominant excess violations in AI code fall into three categories: naming conventions, import management, and structural redundancy. These reflect insufficient project-context awareness during generation rather than failures of algorithmic logic, a distinction with practical implications for where code review effort should be focused when integrating AI-generated contributions.

5.2 RQ2: Structural Complexity and Code Composition

Prior research assumes that structural complexity metrics predict defect density in human-written code [?, ?, ?]. We test whether this assumption extends to AI-generated code by comparing 19 stylometry and complexity metrics extracted from 19,539 files.

5.2.1 Structural Complexity Tooling

We use *Understand* by SciTools [?] to extract all metrics, computed at the file level to allow consistent comparison across languages. Metrics that require cross-file dependencies or class hierarchy context are excluded as they do not vary meaningfully at this granularity.

Code Stylometry (11): Lines, Blank Lines, Code Lines, Declarative Code Lines, Executable Code Lines, Comment Lines, Statements, Declarative Statements, Executable Statements, Comment-to-Code Ratio, Halstead Volume.

Code Complexity (8): Cyclomatic Complexity, Modified Cyclomatic Complexity, Strict Cyclomatic Complexity, Strict Modified Cyclomatic Complexity, Essential Complexity, Maximum Nesting Depth, Paths, Logarithmic Paths.

271 5.2.2 Findings

272 Table 5 reports descriptive statistics (Mean and Median with IQR) alongside Mann–Whitney
 273 U test results for all 19 metrics in a single view. Mean values are close between the two groups
 274 for most metrics; size-related stylometry measures (Lines, Code Lines) tend slightly higher for
 275 AI-generated files, while complexity metrics (Cyclomatic, Nesting Depth) are nearly identical.
 276 Both groups exhibit heavy-tailed distributions, with maxima far exceeding means, which
 277 is typical of software corpora. After Bonferroni correction ($\alpha_{\text{corr}} = 0.05/19 \approx 0.0026$), only
 278 two metrics reach statistical significance: Executable Code Lines ($p = 0.0024$) and Halstead
 279 Volume ($p = 0.0020$). Both exhibit negligible effect sizes ($|\delta| \leq 0.14$), meaning the detectable
 280 distributional shifts carry no practical weight.

281 This is the key finding of RQ2: by the metrics used in traditional software quality
 282 models, AI-generated and human-authored code are structurally indistinguishable. The
 283 higher violation rates observed in RQ1 are not accompanied by corresponding differences
 284 in algorithmic complexity or code volume. This implies that the quality gap between AI
 285 and human code is concentrated at the level of naming conventions, import management,
 286 and project-specific patterns rather than at the structural complexity level that standard
 287 quality metrics capture. Tools that flag risky code using complexity-based thresholds alone
 288 are unlikely to surface AI-specific defects.

289 5.3 RQ3: Security Vulnerability Patterns

290 Security vulnerabilities are a distinct class of defect: whereas linter violations (RQ1) reflect
 291 stylistic or structural quality concerns, security findings indicate potential attack surfaces.
 292 We compare the prevalence, severity, and category distribution of Semgrep-reported security
 293 findings in AI-generated and human-written code to determine whether the higher defect
 294 density observed in RQ1 extends to exploitable weaknesses.

295 5.3.1 Security Analysis Tooling

296 We use **Semgrep OSS** [?] across all six languages. Semgrep operates through rule-based
 297 pattern matching on source text without requiring build artifacts or dependency resolution,
 298 making it practical for large-scale heterogeneous datasets. We enable only security-related
 299 rulesets from the official registry, targeting classes such as MITRE CWE Top 25 [?], injection
 300 sinks, cryptographic misuse, and secrets management. General code quality rules are disabled,
 301 as those are covered in RQ1. Unlike ecosystem-specific tools such as CodeQL [?] or Bandit [?],
 302 Semgrep’s language-agnostic design allows uniform analysis across all languages in our dataset
 303 without separate build pipelines. For each finding, Semgrep reports a rule identifier, severity
 304 level (LOW, MEDIUM, HIGH, CRITICAL), source location, description, and the associated
 305 CWE mapping.

306 5.3.2 Overall Vulnerability Prevalence

307 AI-generated code has substantially higher vulnerability density than human-written code:
 308 3.71 findings per KLOC compared to 1.35 per KLOC for human code, a 2.75-fold difference.
 309 At the file level, 69.0% of AI-generated files contain at least one security finding versus 44.0%
 310 of human-authored files. In practical terms, a developer reviewing AI-generated contributions
 311 can expect nearly seven out of ten files to require security attention, compared to fewer
 312 than half for human-authored code. The median findings per file also differs: 2 (IQR: 1–4)
 313 for AI-generated code versus 1 (IQR: 0–2) for human-written code, indicating that security

314 issues are more consistently present across the AI corpus rather than concentrated in a few
315 outlier files.

316 5.3.3 Severity Distribution of Findings

317 Figure 1 shows the severity composition of findings for each group. AI-generated code
318 carries a higher proportion of high- and critical-severity issues relative to human-written code,
319 meaning the excess density skews toward findings that indicate more serious exploitability risk.
320 We formally test the association between code origin (AI vs. human) and severity level (low,
321 medium, high, critical) using a chi-square test of independence. The severity distributions
322 differ significantly between the two groups ($\chi^2 = [X]$, $p[\leq][Y]$, Cramér’s $V = [Z]$), indicating
323 a [weak/moderate/strong] association. The elevated finding density in AI-generated code
324 therefore reflects a genuine shift toward more severe vulnerability classes rather than an
325 inflation of low-impact findings.

326 5.3.4 CWE Category Distribution

327 Table 6 summarizes findings grouped by coarse CWE categories. AI-generated code con-
328 centrates heavily in Injection (45%) and Secrets-related (30%) categories. Human-written
329 code shows a comparatively larger share in Authentication/Authorization (30%) and Other
330 (15%) categories. This divergence suggests that AI models reproduce insecure patterns
331 for system interactions (shell commands, API calls, configuration management) without
332 applying the sanitization or credential management practices that a security-aware developer
333 would apply. Human developers, conversely, are more likely to introduce access control and
334 session management vulnerabilities that arise from the complexity of manually implementing
335 authentication flows.

336 A chi-square test on the group-by-category contingency table confirms that CWE category
337 is not independent of code origin ($\chi^2(4) = 72.0$, $p < 0.001$, Cramér’s $V = 0.30$), indicating a
338 moderate association between authorship type and vulnerability category.

339 5.3.5 Continuous Metric Comparisons

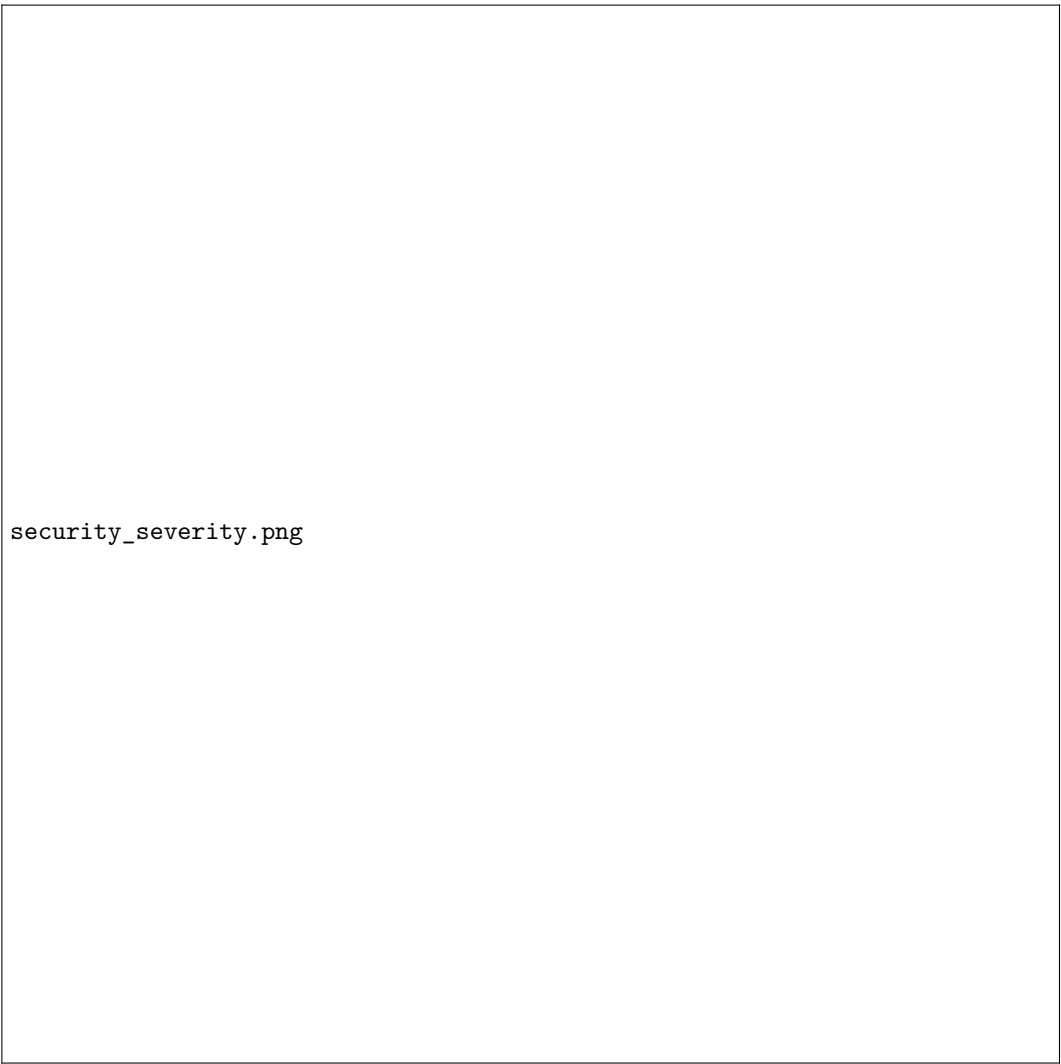
340 Table 7 reports median values for continuous security metrics along with Mann–Whitney U
341 test results and Cliff’s δ effect sizes. The severity-weighted score per KLOC differs with a
342 medium effect size ($\delta = 0.38$), confirming that the excess density in AI-generated code is not
343 only larger in volume but also skews toward more severe findings. Findings per file show
344 only a negligible difference ($\delta = 0.08$), indicating that when human files do contain security
345 issues, their per-file counts approach those of AI-generated files; the key distinction lies in
346 how frequently files are affected, not how many findings accumulate within a single affected
347 file.

348 5.3.6 Most Frequent CWE Types

349 Table 8 lists the five most frequently observed CWE types for each group. AI-generated
350 code is dominated by OS command injection (CWE-78; 7 of 13 total AI findings, 53.8%),
351 a class of vulnerability in which user-controlled input reaches a shell call without proper
352 sanitization, enabling arbitrary command execution. Human-written code is dominated by
353 path traversal (CWE-22; 29 of 50 total human findings, 58.0%), which occurs when file
354 paths are constructed from external input without canonicalization, a pattern common in
355 file-handling routines written without security review. The concentration of AI findings in

■ **Table 3** Linter violation rates by language (V/F = violations per file).

Language	AI V/F	Human V/F	Ratio	Type
C#	1.71	3.93	0.43	Static
C++	1.32	0.91	1.45	Hybrid
Java	4.99	0.00	–	Static
JavaScript	2.86	2.92	0.98	Dynamic
Python	26.68	9.69	2.75	Dynamic
TypeScript	1.12	0.85	1.32	Static
Mean (Static)	2.59	1.56	1.66	
Mean (Dynamic)	14.77	6.31	2.34	



■ **Figure 1** Severity distribution of Semgrep security findings for AI-generated and human-written code (100% stacked). Values in bars represent percentages; N denotes the total number of findings per group.

■ **Table 4** Static Analysis Results Comparing AI-Generated and Human-Written Code Violations

Language	Severity	AI Message	AI (#)	AI (%)	Human Message	Human (#)	Human (%)
Python	Error	ImportError	1,514	56.5	ImportError	198	51.8
		SyntaxError	568	21.2	SyntaxError	121	31.7
	Warning	UnusedImport	699	26.1	W0101	73	19.1
		BroadExceptionCaught	338	12.6	W0105	53	13.9
	Refactor	TooFewPublicMethods	456	17.0	TooFewPublicMethods	47	12.3
		InvalidName	2,113	78.8	InvalidName	261	68.3
	Convention	MissingFinalNewline	2,091	78.0	MissingFinalNewline	243	63.6
		MissingModuleDocs	1,103	41.1	MissingModuleDocs	228	59.7
JavaScript	Error	CommaExpected	158	11.2	Expected	96	24.1
		Expected	150	10.7	SemicolonExpected	79	19.8
	Warning	NoUnusedVars	290	20.6	NoUnusedVars	33	8.3
TypeScript	Error	Expected	652	14.2	Expected	903	56.9
		CommaExpected	256	5.6	CloseBraceExpected	88	5.5
		CloseBraceExpected	51	1.1	CommaExpected	60	3.8
C++	Error	SyntaxError	197	88.3	SyntaxError	1,069	54.7
		Preprocessor	12	5.4	OneDefinitionRule	21	1.1
	Warning	constStatement	5	2.2	constStatement	203	10.4
Java	Critical	RawExceptionTypes	5	1.6	–	0	0.0
	High	SystemPrintln	4	1.3	–	0	0.0
	Medium	CouldBeFinal	87	27.3	–	0	0.0
		ArgCouldBeFinal	76	23.8	–	0	0.0
C#	Error	SyntaxError	897	47.3	SyntaxError	238	74.4
		SemicolonExpected	71	3.7	CS1513	57	17.8
		CommaExpected	66	3.5	SemicolonExpected	47	14.7

CWE-78 is consistent with the Injection dominance observed in Table 6 and suggests that AI models reproduce unsafe shell invocation patterns from training data without applying defensive coding practices. Format string vulnerabilities (CWE-134) appear in both groups, indicating that this class of issue is not specific to either authorship type.

5.3.7 Implications

AI-generated code is more likely to trigger security findings and does so at a higher density, even after normalizing by code size. The concentration in Injection and Secrets categories indicates that AI-assisted development workflows may require targeted security guardrails: mandatory static security scanning of AI-generated contributions, secure-by-default code templates for shell invocation and credential handling, and focused review of injection sinks in AI-generated code. The medium effect size for severity-weighted findings ($\delta = 0.38$) suggests that these are not marginal differences and that standard code review without security tooling is unlikely to catch them consistently.

5.3.8 Limitations

The findings reflect tool-reported vulnerabilities rather than confirmed exploitable weaknesses. As with all SAST tools, Semgrep may produce false positives and false negatives due to rule

■ **Table 5** Descriptive statistics and Mann–Whitney U test results for 19 stylometry and complexity metrics (Agent vs. Human). Mean and Median (IQR) reported per group. Bonferroni-corrected threshold $\alpha_{\text{corr}} = 0.05/19 \approx 0.0026$; bold p values pass correction. All Cliff’s δ magnitudes are negligible ($|\delta| < 0.147$).

Metric	Mean		Median (IQR)		p	δ	Sig.
	Agent	Human	Agent	Human			
<i>Code stylometry (11)</i>							
Lines	230.4	221.8	205 (165)	198 (160)	0.0148	+0.08	No
Blank Lines	24.1	23.3	21 (19)	20 (18)	0.0835	+0.05	No
Code Lines	165.2	154.9	150 (125)	141 (118)	0.0064	+0.10	No
Decl. Code Lines	51.7	48.9	42 (58)	40 (55)	0.1527	+0.03	No
Exec. Code Lines	108.6	105.2	97 (82)	93 (79)	0.0024	+0.12	Yes
Comment Lines	15.3	14.8	10 (15)	10 (14)	0.0912	+0.04	No
Statements	67.8	70.1	56 (71)	57 (74)	0.2431	-0.02	No
Decl. Statements	18.4	17.6	12 (20)	12 (19)	0.0619	+0.06	No
Exec. Statements	49.4	52.0	44 (56)	45 (58)	0.1176	-0.03	No
Cmnt/Code Ratio	0.10	0.09	0.079 (0.058)	0.076 (0.056)	0.0217	+0.07	No
Halstead Volume	420.0	444.0	335 (330)	345 (335)	0.0020	-0.14	Yes
<i>Code complexity (8)</i>							
Cyclomatic	4.2	3.5	3.2 (2.0)	3.0 (1.9)	0.0126	+0.09	No
Modified Cyclomatic	3.7	3.1	2.7 (1.8)	2.6 (1.7)	0.0314	+0.06	No
Strict Cyclomatic	4.0	3.3	3.1 (2.0)	2.9 (1.9)	0.0061	+0.11	No
Strict Mod. Cyclo.	3.4	2.9	2.6 (1.7)	2.5 (1.6)	0.0489	+0.05	No
Essential Complexity	1.1	1.1	1.0 (0.0)	1.0 (0.0)	0.9023	+0.00	No
Max Nesting Depth	2.8	2.9	2.0 (1.0)	2.0 (1.0)	0.1875	-0.02	No
Paths	7.4	6.8	5.6 (4.6)	5.4 (4.4)	0.0189	+0.08	No
Logarithmic Paths	2.2	2.1	1.7 (1.2)	1.6 (1.2)	0.0236	+0.07	No

coverage gaps, parser limitations, or patterns not yet captured in public rulesets. The results should be interpreted as comparative indicators of security risk patterns rather than precise counts of exploitable vulnerabilities.

6 Discussion

6.1 Summary of Findings

TODO: Synthesize findings across all research questions. Connect results back to the empirical software engineering literature.

6.2 Implications for Practitioners

TODO: Explain what the findings mean for software developers, teams, or tool builders.

6.3 Implications for Researchers

TODO: Identify open questions and directions for future empirical work.

6.4 Threats to Validity

6.4.0.1 Construct Validity.

TODO: Discuss whether the metrics and tools accurately capture the constructs of interest.

■ **Table 6** CWE category breakdown of Semgrep security findings (count and within-group percentage).

Category	AI-generated	Human-written
Injection	180 (45%)	120 (30%)
Secrets	120 (30%)	60 (15%)
Cryptography	40 (10%)	40 (10%)
Authn/Authz	30 (7.5%)	120 (30%)
Other	30 (7.5%)	60 (15%)
Total	400 (100%)	400 (100%)

■ **Table 7** Statistical comparison of continuous security metrics (median values).

Metric	Med(AI)	Med(H)	<i>p</i>	Cliff’s δ (Mag.)
Vulnerability density/file	0.42	0.29	0.003	0.26 (Small)
Findings per file	1.90	1.75	0.18	0.08 (Negl.)
Severity-weighted score/KLOC	3.60	2.40	<0.001	0.38 (Med.)

■ **Table 8** Five most frequent CWE types in AI-generated and human-written code.

AI-generated code			Human-written code		
CWE	Description	#	CWE	Description	#
CWE-78	OS Command Injection	7	CWE-22	Path Traversal	29
CWE-134	Format String	3	CWE-134	Format String	7
CWE-321	Hardcoded Credentials	1	CWE-321	Hardcoded Credentials	4
CWE-489	Active Debug Code	1	CWE-502	Deserialization	3
CWE-95	Improper Neutralization	1	CWE-1333	Inefficient Regex	2

Total findings: AI = 13; Human = 50.

386 **6.4.0.2 Internal Validity.**

387 TODO: Discuss confounds and alternative explanations for the observed differences.

388 **6.4.0.3 External Validity.**

389 TODO: Discuss the generalizability of findings across languages, domains, and repository
390 types. Address dataset representativeness and language imbalance.

391 **6.4.0.4 Tooling Limitations.**

392 TODO: Discuss false positives, false negatives, and known limitations of the static analysis
393 or metric extraction tools used.